

E: Custom Keypad

Teclado personalizado

Nombre del archivo fuente (*Source file name*): customkeypad.c, customkeypad.cpp, customkeypad.java o customkeypad.py

Autor (*Author*): Edwin Niño

Los teclados de los primeros teléfonos móviles tenían teclas numéricas que cumplían una doble función para la entrada de texto. En los sistemas clásicos de entrada de texto por pulsación múltiple (*multi-tap*) (y en las distribuciones predecesoras de sistemas predictivos como T9), las 26 letras del alfabeto romano se repartían entre un número reducido de botones físicos. Típicamente se usaban 8 botones (rotulados del 2 al 9), con 3 o 4 letras asignadas a cada tecla en orden alfabético (por ejemplo, la tecla 2 correspondía a abc, la tecla 3 a def, y así sucesivamente).

Para escribir una letra específica en un teclado así, el usuario debe presionar repetidamente el botón correspondiente: presionar el botón una vez produce su primera letra, presionarlo dos veces produce su segunda letra, presionarlo tres veces produce su tercera letra, y así sucesivamente. Por ejemplo, si la tecla 2 contiene abc, escribir 'a' requiere 1 pulsación, escribir 'b' requiere 2 pulsaciones y escribir 'c' requiere 3 pulsaciones.

Un fabricante de hardware especializado diseña dispositivos móviles personalizados para diferentes mercados lingüísticos y aplicaciones de dominios específicos. Como la eficiencia al escribir depende en gran medida de las frecuencias de las letras del idioma de destino, el equipo de ingeniería evalúa cada distribución del teclado con un corpus de texto exhaustivo que sirve como muestra representativa de ese idioma. Dados un texto de muestra representativo (*representative sample text*) de ese tipo y el número de botones de letras disponibles B ($1 \leq B \leq 26$), tu tarea es particionar las 26 letras del alfabeto romano (de 'a' a 'z') entre los B botones de modo que el número total de pulsaciones de botón necesarias para escribir todas las letras del texto de muestra sea mínimo.

El diseño del teclado debe cumplir las siguientes reglas:

- Cada una de las 26 letras romanas (de 'a' a 'z') debe asignarse a exactamente un botón.
- Cada uno de los B botones (numerados de 1 a B) debe tener asignada al menos una letra.
- La partición debe preservar el orden alfabético entre los botones: todas las letras asignadas al botón i deben ser alfabéticamente menores que (preceder a) todas las letras asignadas a los botones siguientes $j > i$ (de forma equivalente, toda letra de un botón debe ser alfabéticamente mayor que todas las letras de los botones anteriores).
- Dentro de cada botón, las letras deben formar un rango alfabéticamente consecutivo y están dispuestas en orden alfabético. Por ejemplo, si $B = 3$, una partición válida podría asignar a-k al botón 1, l-n al botón 2 y o-z al botón 3.
- El costo de escribir una letra es su posición (contando desde 1) dentro del botón al que está asignada.
- Aunque el texto de muestra puede contener caracteres de espacio, los espacios se introducen mediante una tecla dedicada aparte (independiente de los B botones de letras) y **no** se cuentan en el costo total de pulsaciones.

Dados el número de botones B y el texto de muestra representativo, determina una partición óptima de las 26 letras entre los B botones que minimice el total de pulsaciones.

Entrada (Input)

La entrada consiste en múltiples casos de prueba.

Cada caso de prueba consta de dos líneas:

- La primera línea contiene un único entero B ($1 \leq B \leq 26$): el número de botones de letras disponibles en el teclado.
- La segunda línea contiene el texto de muestra representativo. El texto no es vacío y consta solo de letras romanas minúsculas (de 'a' a 'z') y espacios (' '). La longitud del texto de muestra no excede 10^5 caracteres.

La entrada termina con una línea que contiene un único cero ($B = 0$). Esta línea no debe procesarse.

La suma de las longitudes de todos los textos de muestra de todos los casos de prueba no excede 10^6 .

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba, imprime una sola línea que describa la partición óptima de las 26 letras en los B botones. La línea debe contener las letras asignadas a cada botón en orden alfabético, separadas por una barra vertical (|).

Si hay varias particiones que logran el mismo costo mínimo de pulsaciones, elige la que coloque los separadores (|) lo más a la derecha posible (es decir, prioriza maximizar la posición del separador más a la derecha; si aún hay empate, la del segundo separador más a la derecha, y así sucesivamente).

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
3 hello world 8 the quick brown fox jumps over the lazy dog 5 icpc colombia 0	abcdefghijklmnopqrstuvwxyz abcd efg hijk lmn opq rst uvw xyz ab cdefgh ijk lmn opqrstuvwxyz

Explicación de los casos de ejemplo (Explanation of the sample cases)

- En el primer caso de prueba ($B = 3$, "hello world"), el costo mínimo es 35 pulsaciones. Tanto `abcdefghijklmnopqrstuvwxyz` como `abcdefghijklmnopqrstuvwxyz` logran este costo mínimo, pero se elige la primera porque su separador más a la derecha (después de 'q') está más a la derecha que el de la alternativa (después de 'n').
- En el segundo caso de prueba ($B = 8$, el pangrama), hay 6 particiones que logran el costo mínimo de 67 pulsaciones; la regla de desempate prioriza los separadores más a la derecha y selecciona `abcd|efg|hijk|lmn|opq|rst|uvw|xyz`.
- En el tercer caso de prueba ($B = 5$, "icpc colombia"), la única partición óptima es `ab|cdefgh|ijk|lmn|opqrstuvwxyz`, que requiere 15 pulsaciones.